

SlickChat — Arquitetura do Sistema

1. Introdução

Este documento descreve a arquitetura do sistema SlickChat, projetada para suportar comunicação em tempo real, processamento assíncrono e escalabilidade horizontal utilizando uma abordagem baseada em eventos.

2. Estilo Arquitetural

O SlickChat adota uma arquitetura **Event-Driven distribuída**.

Características principais:

- comunicação em tempo real via WebSocket
- processamento assíncrono baseado em eventos
- desacoplamento entre componentes
- escalabilidade horizontal

A comunicação entre componentes ocorre através de um **broker de mensageria (Kafka)**.

Além disso, o sistema utiliza **Redis** para dados temporários e operações de baixa latência.

3. Componentes Principais

```
Edge Router (Traefik)
Web Client
Realtime Gateway
API Service
Kafka Event Stream
Workers
Redis
Postgres Database
Media Storage (MinIO)
```

4. Edge Router (Traefik)

O **Edge Router** é o ponto de entrada único para todo o tráfego externo. Utilizamos **Traefik** como reverse proxy e load balancer.

Funções:

- Terminação TLS (HTTPS/WSS).
 - Roteamento baseado em path:
 - `/api/*` → API Service
 - `/socket/*` → Realtime Gateway
 - Balanceamento de carga entre instâncias dos serviços.
 - Fornecimento de certificados SSL automáticos (Let's Encrypt).
-

5. Realtime Gateway

O **Realtime Gateway** é responsável pela comunicação em tempo real com os clientes.

Funções:

- gerenciar conexões WebSocket
- autenticar usuários (via Redis)
- gerenciar sessões ativas
- atualizar presença de usuários (Redis)
- receber eventos do cliente
- publicar eventos no Kafka
- distribuir mensagens recebidas para clientes conectados (via Redis Pub/Sub)

O gateway é **stateless**, permitindo múltiplas instâncias.

Nota de Implementação: Para viabilizar a escalabilidade horizontal, o Message Fanout Worker resolve membros e conexões no Redis e publica os eventos em canais `connection:<connection_id>`. O Realtime Gateway com a conexão ativa consome esse canal via Redis Pub/Sub e entrega ao cliente local.

6. API Service

O **API Service** fornece endpoints HTTP para funcionalidades administrativas e operações que não exigem comunicação em tempo real.

Exemplos:

- criação de conta
- autenticação
- gerenciamento de salas
- moderação
- upload de anexos (geração de pre-signed URLs)

Este serviço também publica eventos no Kafka quando necessário.

7. Broker de Mensageria (Kafka)

O SlickChat utiliza **Kafka** como broker de eventos.

Responsabilidades:

- transporte de eventos entre serviços
- desacoplamento entre componentes
- suporte a alta taxa de mensagens
- buffer de eventos para processamento assíncrono

Eventos do sistema (catálogo alvo; implementação atual prioriza fluxo de mensagens em **message-events**):

User:

```
UserCreated, UserAuthenticated, UserIdentityRotated  
UserConnected, UserDisconnected, UserPresenceChanged  
UserSessionCreated, UserSessionExpired
```

Room:

```
RoomCreated, RoomExpired, UserJoinedRoom, UserLeftRoom
```

Message:

```
MessageSent, MessageDelivered, MessageRead, MessageDeleted, MessageExpired
```

Attachment:

```
AttachmentUploaded
```

Moderation:

```
UserMuted, UserKicked, UserBanned, ReportCreated
```

O Kafka permite que múltiplos consumidores processem eventos de forma independente.

8. Redis

Redis é utilizado para armazenar dados temporários que exigem acesso rápido.

Principais responsabilidades:

- presença de usuários (online/offline/invisible)
- armazenamento de sessões ativas (token → user_id)
- controle de rate limiting (global)
- cache de dados frequentemente acessados
- pub/sub para entrega de mensagens aos gateways

9. Workers

Workers são responsáveis pelo processamento assíncrono de eventos consumidos do Kafka.

Worker	Responsabilidade
Message Fanout	Consome eventos do Kafka e publica fanout em canais Redis por conexão (<code>connection:*</code>).
Persistence	Persiste mensagens e eventos no Postgres (exceto quando <code>is_zero_logging = true</code>).
TTL	Varre mensagens expiradas no Postgres e publica <code>message.expired.v1</code> no Kafka.
Moderation	Processa ações de moderação (bans, mutes, kicks) e atualiza permissões.

Workers podem ser escalados horizontalmente.

Para viabilizar a escalabilidade horizontal, o Message Fanout Worker consulta membros/conexões no Redis e publica eventos por conexão em `connection:<connection_id>`. O gateway assinante desse canal encaminha a mensagem ao cliente conectado.

10. Persistência de Dados (Postgres)

O sistema utiliza **Postgres** para armazenamento persistente.

Dados armazenados:

- usuários
- salas
- mensagens (apenas quando persistência normal ou TTL)
- membros de salas
- ações de moderação
- denúncias

11. Estratégia de Mensagens Efêmeras

O sistema suporta três modos de persistência:

Modo	Comportamento
Persistência normal	Mensagens são armazenadas permanentemente.
TTL	Mensagens são armazenadas temporariamente e removidas após expiração.
Zero Logging Mode	Mensagens não são persistidas; existem apenas no fluxo de eventos e memória.

No **Zero Logging Mode**, o **Persistence Worker** descarta o evento imediatamente.

12. Armazenamento de Mídia (MinIO)

Arquivos anexados são armazenados no **MinIO**.

O fluxo de upload:

1. Cliente solicita upload à **API Service**.
2. API gera uma **pre-signed URL** no MinIO.
3. Cliente faz **PUT** diretamente para o MinIO.
4. Cliente ou MinIO notifica a API para registrar o metadado no Postgres.

13. Escalabilidade

Componentes escaláveis horizontalmente:

- Edge Router
- Realtime Gateway
- Workers
- API Service

Kafka distribui eventos entre múltiplas instâncias consumidoras.

Redis pode ser executado em cluster.

14. Segurança

Principais medidas:

- autenticação por senha, recovery key ou identity key
- hashing de credenciais
- rate limiting global (Redis)
- moderação de salas

- comunicação segura (HTTPS/WSS)
- ausência de coleta de dados pessoais

15. Observabilidade

Logs estruturados para monitoramento:

- eventos de mensagens
- conexões WebSocket
- erros do sistema
- ações de moderação

Ferramenta: Prometheus